UD3.1. Funciones de usuario y del lenguaje

Introducción al uso de Python

Resultado de aprendizaje y criterios de evaluación

RA3. Aplica conceptos de programación orientada a objetos para crear código reutilizable.

CEa). Se han utilizado funciones incorporadas del lenguaje.

CEb). Se han definido funciones propias.

Introducción

¿Qué es una función?

Una función es un bloque de código reutilizable y modular, que aplica el principio DRY: **D**on't **R**epeat **Y**ourself.

Hay dos tipos de funciones:

- **Funciones del lenguaje**
- **Funciones del usuario**

Sintaxis básica

Las funciones se declaran en Python empleando la palabra clave **def**

```
def funcion():
    # código de la función
```

Los paréntesis indicarán los parámetros o argumentos de la función. Una función puede no tener parámetros.

Los dos puntos indicarán el comienzo de la función. El cuerpo de la función deberá ir tabulado respecto a su definición.

Sintaxis básica

Las funciones pueden devolver un valor o no devolver nada. En el segundo caso suelen denominarse procedimientos, aunque la sintaxis sigue siendo similar.

Una función que retorne algún valor usará la cláusula return al final de la misma.

```
def funcion():
    # código de la función
    return valor
```

Parámetros

A la hora de indicar los parámetros de una función, podemos elegir si decidir el tipo de dichos parámetros o si dejarlo implícito.

```
def sumar(a, b):
    return a + b


def sumar(a: int, b: int):
    return a + b
```

Parámetros

Los parámetros, ya indiquemos su tipo o no, pueden tener valores por defecto.

```
def sumar(a = 1, b = 1):
    return a + b


def sumar(a: int = 1, b: int = 1):
    return a + b
```

Valor de retorno

Al igual que podemos decidir si indicar el tipo de los parámetros, también podemos decidir si indicar el tipo de retorno.

```
def sumar(a, b) -> int:
    return a + b


def sumar(a: int, b: int) -> int:
    return a + b
```

Retornos múltiples

A diferencia de en la mayoría de lenguajes, en Python podemos retornar varios valores en una función.

```
def suma(a, b):

    return a+b, a, b
```

Parámetros indeterminados

Podemos, en lugar de pasar un número definido de parámetros, pasar un número indefinido de parámetros a la función.

Para ello usaremos el modificador *

```
def suma(*numeros):
    return sum(numeros)
```

Parámetros indeterminados

En el ejemplo anterior, *numeros sería una tupla, por lo que podemos recorrerla con un bucle.

```
def suma(*numeros):
    suma = 0
    for n in numeros:
        suma += n
        return suma
```